JavaScript

JavaScript es un lenguaje de programación del lado del cliente web (es decir, se ejecuta en nuestro ordenador, no en el servidor, implementado en el navegador web) permite cambiar el contenido o el estilo de las etiquetas de los elementos HTML creando efectos atractivos y dinámicos en las páginas web.

JavaScript puede ser colocado tanto en las secciones **<body>** como **<head>** de una página HTML

La etiqueta <script>

En HTML, el código JavaScript debe ser insertado entre las etiquetas **<script>** y **</ script>.**

```
<script>
document.getElementById("demo").innerHTML = "Esta es una instrucción JS";
</script>
```

JavaScript puede cambiar el contenido HTML

Uno de los muchos métodos JavaScript HTML es **getElemenById ()** . En este ejemplo utiliza el método de "encontrar" un elemento HTML (con id = "demo") y cambiar el contenido del elemento (**innerHTML**) .

```
document.getElementById("demo").innerHTML ="Contenido de la etiqueta";
```

```
<!DOCTYPE html>
<html>
<body>
<h1>Empezamos el lenguaje JavaScript</h1>
<p id="demo">JavaScript puede cambiar el contenido HTML</p>
<button type="button" onclick="document.getElementById('demo').innerHTML =’Contenido de la etiqueta cambiado’">¡Pulsa aquí!</button>
</body>
</html>
```

JavaScript puede cambiar los atributos HTML

En este ejemplo se cambia una imagen, modificando el atributo src de una etiqueta <**img**>:

```
<!DOCTYPE html>
<html>
<head><meta charset="iso-8859-1"></head>
<body>
<h1>Cambiar atributo mediante JS</h1>
<a id="enlace" href="https://www.filmaffinity.com/">Enlace</a></br></br>
<button type="button" onclick="document.getElementById('enlace').href ='http://www.elperiodicoextremadura.com/'">¡Cambiar a un periódico!</button>
</body></html>
```

JavaScript puede cambiar los estilos HTML (CSS)

Cambiar el estilo de un elemento HTML, es una variante de cambiar un atributo HTML.

```
<!DOCTYPE html>
<html>

<head>

<style>

#demo {color:red}

</style>

</head>

<body>

<h1> Cambiar estilo con JS</h1>

<p id="demo">Párrafo que puede cambiar el color con un click de ratón</p>

<button type="button" onclick="document.getElementById('demo').style.color='green'">!Haz
Click!</button>

</body></html>
```

JavaScript puede ocultar elementos HTML

La ocultación de elementos HTML se puede hacer cambiando el estilo de visualización:

```
document.getElementById('demo').style.display = 'none'
```

Funciones de JavaScript y Eventos

Una **función** JavaScript es un bloque de código JavaScript que se ejecuta cuando se hace una llamada. Por ejemplo, una función puede ser ejecutada cuando un **evento** ocurre, como cuando el usuario hace click en un botón. Algunos eventos:

EVENTO	SIGNIFICADO
onclick	Hacer click sobre un elemento
ondblclick	Hacer doble click sobre un elemento
onchange	Un elemento HTML se ha cambiado
onkeydown	El usuario presiona una tecla del teclado
onload	El navegador ha terminado de cargar la página
onmouseover	El usuario mueve el ratón sobre un elemento HTML.
onmouseout	El usuario aleja el ratón de un elemento HTML.

```
<!DOCTYPE html>
<html>
<head>
<script
function myFunction() {
  document.getElementById("demo").innerHTML = "Cambiar el párrafo.";}
```

```
</script>
</head>

<body>

<h1>Encabezado</h1>

<p id="demo">Un párrafo</p>

<button type="button" onclick="myFunction()">¡Probamos!</button>

</body>
</html>
```

El DOM HTML (Document Object Model)

Cuando se carga una página web, el navegador crea una DOM (Modelo del Objeto del Documento) de la página.
Con el DOM, JavaScript puede acceder y cambiar todos los elementos de un documento HTML
Los ***métodos*** DOM HTML son las **acciones** que se pueden realizar en los elementos HTML
Las ***propiedades*** DOM HTML son los **valores** de los elementos HTML que se pueden ajustar o cambiar.

```
<html>
<body>

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = "Hola Ciudad Real!";
</script>

</body>
</html>
```

En el ejemplo anterior, getElementById() es un **método**, mientras que innerHTML es una **propiedad**.

Métodos para encontrar elementos HTML

Method	Description
document.getElementById(*id*)	Find an element by element id
document.getElementsByTagName(*name*)	Find elements by tag name
document.getElementsByClassName(*name*)	Find elements by class name

Ejemplos:

```
<script>
document.getElementById("demo")
document.getElementsTagName("h1")
document.getElementsClassName("link")
</script>
```

Métodos para modificar elementos HTML

Method	Description
element.innerHTML = *new html content*	Change the inner HTML of an element
element.attribute = new value	Change the attribute value of an HTML element
element.setAttribute*(attribute, value)*	Change the attribute value of an HTML element
element.style.*property = new style*	Change the style of an HTML element

document.getElementById("demo").innerHTML="Bienvenido a JavaScript"
// esta sentencia es igual a las dos siguientes

```
<script>
x=document.getElementById("demo");
x.innerHTML="Bienvenido a JavaScript";
</script>
```

Métodos para añadir y eliminar elementos HTML

Method	Description
document.createElement(*element*)	Create an HTML element
document.removeChild(*element*)	Remove an HTML element
document.appendChild(*element*)	Add an HTML element
document.replaceChild(*element*)	Replace an HTML element
document.write(*text*)	Write into the HTML output stream

Crear un elemento HTML:

```
…
<button onclick="myFunction()">Inténtalo</button>
<script>
function myFunction() {
  var btn = document.createElement("BUTTON"); //crear un elemento vacio
  btn.setAttribute("id", "demo"); //añadir el atributo id con el valor demo al elemento btn
  var t = document.createTextNode("BOTÓN CREADO");  crear un contenido
  btn.appendChild(t); //añadir el contenido al elemento btn
  document.body.appendChild(btn); añadir el elemento btn al body
}
</script>
…
```

Cambiar el estilo de un elemento HTML, es una variante de cambiar un atributo HTML:

```
<script>
function myFunction() {
var x = document.getElementById("demo");
x.style.fontSize = "25px";
x.style.color = "red";
}
</script>
<button type="button" onclick="myFunction()">Haz click</button>
```

En este ejemplo, una función de JavaScript se coloca en la sección **<body>** de una página HTML.

```
<!DOCTYPE html>
<html>
<body>
<h1>My Web Page</h1>
<p id="demo">A Paragraph</p>
<button type="button" onclick="myFunction()">Try it</button>
<script>
function myFunction() {
  document.getElementById("demo").innerHTML = "Paragraph changed.";
}
</script>
</body></html>
```

Para utilizar un script externo, poner el nombre del archivo de script en el atributo **src** de la etiqueta <script>:

```
<!DOCTYPE html>
<html>
<body>
<h1> JavaScript externa</h1>
<p id="demo">Un párrafo.</p>
<button type="button" onclick="myFunction()">Inténtalo</button>
<p>Nota: myFunction se almacena en un fichero externo llamado "myScript.js".</p>
<script src="myScript.js"></script>
</body></html>
```

Salida

Se puede utilizar la **ventana de alerta** para mostrar los datos

```
<!DOCTYPE html>
<html>
<body>
<h1>Mi primera página web.</h1>
<p>Mi primer párrafo.</p>

<script>
alert(5 + 6);
</script>
</body>
</html>
```

También se utiliza document.write ()

```
<!DOCTYPE html>
<html>
<body>
<h1> Mi primera página web.</h1>
<p> Mi primer párrafo.</p>
<script>
document.write (5 + 6);
</script>
</body></html>
```

Mi primera página web.

Mi primer párrafo.

11

Utilizando documen.write() después de que un documento HTML se ha cargado completamente se elimina todo el documento HTML existente

```
<!DOCTYPE html>
<html>
<body>
<h1> Mi primera página web </h1>
<p> Mi primer párrafo.</p>
<button type="button" onclick="document.write(5 + 6)">Inténtalo</button>
</body>
</html>
```

Mi primera página web

Mi primer párrafo.

Intentalo

11

Sintaxis de JS

La **sintaxis** es un conjunto de reglas para construir un programa en JavaScript.

Un **programa** es un conjunto de instrucciones ejecutadas en un ordenador

En un **lenguaje de programación**, estas instrucciones de programa se llaman **sentencias**.
JavaScript es un lenguaje de programación.

Las **Sentencias** de JavaScript están separados por **puntos y comas**

```
<!DOCTYPE html>
<html>
<body>
<h1> Sentencias en JavaScript </h1>
<p>Las instrucciones se separan por puntos y comas.</p>
<p>Las variables x, y, z se asignan a los valores 5, 6 y 11</p>
<p id="demo"></p>
<script>
var x = 5;
var y = 6;
var z = x + y;
document.getElementById("demo").innerHTML = z;
</script>
</body>
</html>
```

declaraciones de JavaScript

Las declaraciones están separados por punto y coma.

Las variables x, y, y z se asignan los valores de 5, 6, y 11:

11

Las **sentencias** en JavaScript se componen: *valores*, *operadores*, *expresiones*, *palabras claves* y *comentarios*.
Hay dos tipos de **valores**

- Fijos llamados **literales**
- **Variables**

Literales

Literales numéricos: se escriben con o sin decimales

```
<!DOCTYPE html>
<html>
<body>
<h1>Numeros de JavaScript</h1>
<p>los números pueden escribirse con o sin decimales</p>
<p id="demo"></p>
<script>
document.getElementById("demo").innerHTML = 10.50;
</script>
```

```
</body>
</html>
```

números de JavaScript

Número puede escribirse con o sin decimales.

10.5

Literales de cadenas de caracteres

```
<script>
document.getElementById("demo").innerHTML = 'John Doe';
</script>
```

JavaScript Cadenas

Las cadenas pueden ser escritos con comillas simples o dobles.

John Doe

Variables

Se utilizan para almacenar valores de datos. JS utiliza la palabra **var** para declarar variables

Un **signo igual** se utiliza para **asignar valores** a las variables.

En este ejemplo, x se define como una variable. Entonces, x se asigna el valor 6

```
<script>
var x;
x = 6;
document.getElementById("demo").innerHTML = x;
</script>
```

Las variables de JavaScript

En este ejemplo, x se define como una variable. Entonces, x se le asigna el valor de 6:

6

Operadores

Los **operadores** son símbolos que indican cómo se deben manipular los valores

JavaScript utiliza un **operador de asignación** (=) para dar valores a las variables:

```
<script>
var x = 5;
var y = 6;
document.getElementById("demo").innerHTML = x + y;
</script>
```

Asignación de valores

En JavaScript del operador = se utiliza para asignar valores a las variables.

11

JS utiliza **operadores aritméticos** (+ - * /) para calcular los valores

```
<script>
document.getElementById("demo").innerHTML = (5 + 6) * 10;
</script>
```

Los operadores de JavaScript

JavaScript utiliza operadores aritméticos para calcular los valores (como álgebra).

110

Expresiones

Una expresión es una combinación de valores, variables y operadores que calcula un valor

```
5 * 10
x* 10
"John" + " " + "Doe"
<script>
document.getElementById("demo").innerHTML = "John" + " "  + "Doe";
</script>
```

Palabras claves de JavaScript

Las **palabras claves** se utilizan para identificar las acciones que se deben realizar.

La palabra clave **var** indica que se crea una variable

```
<script>
var x = 5 + 6;
var y = x * 10;
document.getElementById("demo").innerHTML = y;
</script>
```

Comentarios en JS

El código o el texto que se escribe después de **//** o **/*** y ***/**son ignorados por el navegador

```
var x = 5;   // Se ejecuta

// var x = 6;   no se ejecuta
```

Identificadores de JavaScript

Los **identificadores** son nombre, se utilizan para las variables, etiquetas, palabras claves y funciones.

El **primer carácter** debe ser una letra, un guión bajo (_) o un signo dólar ($). Los siguientes caracteres pueden ser letras, dígitos, guiones o signos de dólar.

```
var $numero1;
var _$letra;
var $$$otroNumero;
```

JavaScript es sensible a mayúsculas

Todos los identificadores en JavaScript son distintos en mayúsculas que en minúsculas .

Las variables **lastName** y **lastname** , son dos variables diferentes.

```
lastName = "Doe";
lastname = "Peterson";
```

Sentencias de JavaScript

Son instrucciones que serán ejecutadas por el navegador

```
document.getElementById("demo").innerHTML = "Hello Dolly.";
```

Programas JavaScript

La mayoría de los programas JS contienen muchas sentencias JS

Las instrucciones se ejecutan, uno por uno, en el mismo orden en que se escriben.

En este ejemplo, x, y, z se dan valores, y, finalmente, se muestra z:

```
<script>

var x = 5;
var y = 6;
var z = x + y;
document.getElementById("demo").innerHTML = z;

</script>
```

El punto y coma (;)separan sentencias JS. Se pueden poner con punto y coma varias sentencias en una línea

```
a = 5; b = 6; c = a + b;
```

JS ignora múltiples espacios. Puede agregar espacio en blanco a su script para que sea más legible. Las siguientes líneas son equivalentes:

```
var person = "Hegel";
var person="Hegel";
```

Si una instrucción JS no cabe en una línea, el mejor lugar para dividirla es después de un operador

```
document.getElementById("demo").innerHTML ="Hello Dolly.";
```

Bloques de código JavaScript

Se pueden agrupar las instrucciones en bloques de código, entre llaves {...}

```
function myFunction() {
    document.getElementById("demo").innerHTML = "Hello Dolly.";
    document.getElementById("myDIV").innerHTML = "How are you?";
}
```

Palabras claves o reservadas

Son las palabras (en inglés) que se utilizan para construir las sentencias de JavaScript y que por tanto no pueden ser utilizadas libremente. Las palabras actualmente reservadas por JavaScript son: **break, case, catch, continue, default, delete, do, else, finally, for, function, if, in, instanceof, new, return, switch, this, throw, try, typeof, var, void, while, with.**

Comentarios de una línea

Cualquier texto entre **//** y el final de la línea, será ignorado por JavaScript (no será ejecutado).

En este ejemplo se utiliza una sola línea de comentario antes de cada línea, para explicar el código:

```
// Change heading:
document.getElementById("myH").innerHTML = "My First Page";
// Change paragraph:
document.getElementById("myP").innerHTML = "My first paragraph.";
```

```
var x = 5;      // Declare x, give it the value of 5
var y = x + 2;  // Declare y, give it the value of x + 2
```

Comentarios multilíneas

Los comentarios multilíneas comienzan con **/ *** y terminan con *** /**.

Cualquier texto entre / * y * / serán ignorados por JavaScript.

En este ejemplo se utiliza un comentario de varias líneas (un bloque de comentario) para explicar el código

```
/*
The code below will change
the heading with id = "myH"
and the paragraph with id = "myP"
in my web page:
*/
```

El uso de comentarios para evitar la ejecución de código, es adecuado para las pruebas de código.

```
//document.getElementById("myH").innerHTML = "My First Page";
document.getElementById("myP").innerHTML = "My first paragraph.";

/*
document.getElementById("myH").innerHTML = "My First Page";
document.getElementById("myP").innerHTML = "My first paragraph.";
*/
```

Las variables de JavaScript

Son contenedores para el almacenamiento de valores de datos.

```
var x = 5;
var y = 6;
var z = x + y;
```

La creación de una variable se llama “declarar” una variable

Se declara una variable de JavaScript con la palabra clave **var**

```
var carName;
```

También puede asignar un valor a la variable cuando se declara.:

```
var carName = "Volvo";
```

```
<p id="demo"></p>
<script>
var carName = "Volvo";
document.getElementById("demo").innerHTML = carName;
</script>
```

Una variable declarada sin un valor tendrá el valor **indefinido**

```
var carName;
```

Al igual que con el álgebra, puede realizar operaciones aritméticas con variables de JavaScript, utilizando operadores como **=** y **+**

```
var x = 5 + 2 + 3;
```

También se pueden agregar cadenas

```
var x = "John" + " " + "Doe";
```

Los operadores aritméticos de JavaScript

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus
++	Increment
--	Decrement

adición

```
var x = 5;
var y = 2;
var z = x + y;
```

El operador + también se puede utilizar para añadir cadenas de texto (concatenar).

```
Var txt1 = "John";
var txt2 = "Doe";
var txt3 = txt1 + " " + txt2;
```

resta

```
var x = 5;
var y = 2;
var z = x - y;
```

multiplicación

```
var x = 5;
var y = 2;
var z = x * y;
```

división

```
var x = 5;
var y = 2;
var z = x / y;
```

módulo(resto de la división)

```
var x = 5;
var y = 2;
var z = x % y;
```

incremento

```
var x = 5;
x++;
var z = x;
```

decremento

```
var x = 5;
x--;
var z = x;
```

Prioridad de los operadores

La multiplicación (*) y división (/) tienen mayor prioridad que la suma (+) y la resta (-)

La prioridad se modifica mediante el uso de paréntesis

```
var x = (100 + 50) * 3;
```

Operadores de asignación de JavaScript

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y

Operadores de comparación y lógicos

==	igual a
===	igual valor e igual tipo
!=	no igual
!==	no igual valor y no igual tipo
>	mayor que
<	menor que
>=	mayor o igual
<=	menor o igual

Tipos de datos de JavaScript

Las **variables** de JS pueden contener muchos **tipos de datos**: numéricos, cadenas, matrices, objetos,...

Cadenas de texto

Las cadenas se escriben con comillas. Puede utilizar comillas simples o dobles.

```
var carName = "Volvo XC60";
var carName = 'Volvo XC60';
```

Puedes usar comillas dobles dentro de las comillas simples y al revés

```
var answer = "It's alright";             // Single quote inside double quotes
var answer = "He is called 'Johnny'";    // Single quotes inside double quotes
var answer = 'He is called "Johnny"';    // Double quotes inside single quotes
```

Longitud de una cadena de texto

La longitud de una cadena se encuentra definida en la propiedad **length**

```
<!DOCTYPE html>
<html>
<body>
<p id="demo"></p>
<script>
var txt = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
txt=txt.length
document.getElementById("demo").innerHTML = txt
</script>
</body></html>
```

Caracteres especiales en una cadena de texto

No obstante, a veces las cadenas de texto contienen tanto comillas simples como dobles. Además, existen otros caracteres que son difíciles de incluir en una variable de texto (tabulador, ENTER, etc.). Para resolver estos problemas, **JavaScript** define un mecanismo para incluir de forma sencilla caracteres especiales y problemáticos dentro de una cadena de texto.

Si se quiere incluir...	Se debe incluir...
Una nueva línea	\n
Un tabulador	\t
Una comilla simple	\'
Una comilla doble	\"
Una barra inclinada	\\

```
var texto1 ='Una frase con \'comillas simples\' dentro';

vartexto2 = "Una frase con \"comillas dobles\" dentro";
```

números

```
var x1 = 34.00;      // Written with decimals
var x2 = 34;         // Written without decimals

Con notación científica
var y = 123e5;       // 12300000
var z = 123e-5;      // 0.00123
```

Puedes utilizar la función de JS **isNaN()** para averiguar si un valor es un número

```
<script>
var x = 100; // "Apple"
document.getElementById("demo").innerHTML = isNaN(x);
</script>
```

booleanos

Tiene dos valores: **true** y**false**

```
var x = true;
var y = false;
```

Podemos utilizar la función **Boolean()** para averiguar si una expresión (o un avariable) es verdadera

```
Boolean(10 > 9)         // returns true
```

O incluso más sencillo

```
(10 > 9)               // also returns true
10 > 9                 // also returns true
```

El valor booleano 0 es falso

```
var x = 0;
Boolean(x);        // returns false
```

El valor booleano de **""** (cadena vacía) es **falsa**:

```
var x = "";
Boolean(x);        // returns false
```

Cualquier otro valor es verdadero

```
var x=6
Boolean(x);        // returns true
```

matrices

Si usted tiene una lista de elementos (una lista de nombres de coche, por ejemplo), el almacenamiento de los coches en las variables individuales podría tener este aspecto:

```
var car1 = "Saab";
var car2 = "Volvo";
var car3 = "BMW";
```

Una matriz puede contener muchos valores bajo un único nombre, y se puede acceder a los valores de referencia mediante un número de índice

Se escriben con corchetes **[]**

```
var cars = ["Saab", "Volvo", "BMW"];
```

Ejemplo para mostrar una matriz dentro de un elemento HTML

```
<p id="demo"></p>
<script>
var cars = ["Saab", "Volvo", "BMW"];
document.getElementById("demo").innerHTML = cars;
</script>
```

Los índices de la matriz empieza por la posición 0, lo que significa que el primer elemento [0], el segundo [1], y así sucesivamente

Esta declaración accede al valor del primer elemento de coches:

```
var name = cars[0];
```

Esta declaración modifica el primer elemento de los coches

```
cars[0] = "Opel";
```

La propiedad **length** de una matriz devuelve la longitud de la misma (número de elementos)

```
<script>
var fruits = ["Banana", "Orange", "Apple", "Mango"];
document.getElementById("demo").innerHTML = fruits.length;
</script>
```

Ejemplo de matriz para visualizar un día de la semana y también todos los días.

```
<html>
<body>
<p id="dias"></p>
<p id="demo"></p>
<script>
var semana= ["lunes", "martes","miércoles","jueves","viernes","sábado","domingo"];
var y=document.getElementById("dias");
y.innerHTML=semana[6];
alert(semana);
</script>
</body>
</html>
```

Otra manera de crear una matriz es a través de un objeto

```
var cars = new Array("Saab", "Volvo", "BMW");
```

objetos

Los objetos son variables que contiene nombres(propiedades) y valores. En **JS** se escriben entre llaves. En JavaScript podemos declarar un objeto de varias maneras:

```
var a = {}; // Objeto vacio
var a = new Object(); // Objeto vacio
var a = {clave1: "valor", clave2: 22};
// En a tenemos a.clave1 y a.clave2 con valores "valor" y 22 respectivamente
```

Propiedades del objeto

Las propiedades del objeto se escriben como **nombre : valor,** separados por comas.

```
var person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};
```

Otra forma de declarar el objeto.

```
var person = new Object(); //objeto vacio
person.firstName=”John”;
person.lastName=”Doe”;
person.age=50;
person.eyecolor=”blue”
```

El objeto persona tiene 4 propiedades: nombre, apellido, edad y color_de_ojos

Acceso a las propiedades del objeto

`nombreobjeto.propiedad`

Ejemplo del objeto persona donde se muestra la edad del individuo

```
<html>
<body>
```

```
<p id="demo"></p>
<script>
var persona= {nombre:"Pedro", primerapellido:" Guerrero", edad:28 };
var x=document.getElementById("demo");
x.innerHTML=persona.edad;
</script></body></html>
```

Métodos de objeto

```
var persona= {
nombre:"Pedro",
primerapellido:" Guerrero",
edad:28,
nombrecompleto : function() {
   return this.nombre + " " + this.primerapellido;
 }
};
```

Acceso a las propiedades del objeto

*nombreobjeto.*metodo*()*

Ejemplo del objeto persona donde se muestra el nombre y el primer apellidos del individuo

```
<html>
<body>
<p id="demo"></p>
<script>
var persona= {nombre:"Pedro",
primerapellido:" Guerrero",
edad:28,
nombrecompleto:function() {
   return this.nombre + " " + this.primerapellido;
 }
};
var x=document.getElementById("demo");
x.innerHTML=persona.nombrecompleto();
</script></body></html>
```

El operador typeof

Se puede utilizar el operador **typeof** para encontrar el tipo de una variable **JS**

```
<html>
<body>
<p id="demo"></p>
<script>
    document.getElementById("demo").innerHTML =
    typeof "john" + "<br>" +
    typeof 3.14 + "<br>" +
```

```
    typeof false + "<br>" +
    typeof [1,2,3,4] + "<br>" +
    typeof {name:'john', age:34};
</script>
</body></html>
```

Indefinido

Una variable sin un valor asignado.

```
<script>
var person;
document.getElementById("demo").innerHTML =
person + "<br>" + typeof person;
</script>
```

Nulo

Variable existe, tiene tipo (objeto) pero no valor

```
<script>
var person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};
person = null;
document.getElementById("demo").innerHTML = typeof person;
</script>
```

indefinido y **null** son iguales en valor pero diferentes en tipo:

Valores vacíos

Una variable de cadena vacía tiene un valor y un tipo. Un valor vacío es diferente al valor indefinido.

```
<script>
var car = "";
document.getElementById("demo").innerHTML =
"El valor es: " +car + "<br>" +"El tipo es:" + typeof car;
</script>
```

Eventos HTML

Suceso que ocurre en un sistema. Eventos pueden ser hacer **clic**, **doble clic** en un botón HTML, **minimizar** una ventana, una página web HTML ha terminado de cargar, un campo de entrada cambia, etc

JavaScript permite ejecutar código cuando se detectan eventos.

HTML permite atributos de controlador de eventos, con **código JavaScript** , que se añade a los elementos HTML.

Este ejemplo, el codigo **JS** cambia el contenido del elemento con id=”demo”

```
<button onclick='document.getElementById("demo").innerHTML=Date()'>The time is?</button>
```

El código **JS**, si tiene varias líneas, es más común ver el atributo de eventos llamando a funciones

```
<!DOCTYPE html>
<html>
<body>
<p>Click the button to display the date.</p>
<button onclick="displayDate()">The time is?</button>
<script>
function displayDate() {
document.getElementById("demo").innerHTML = Date();
}
</script>
<p id="demo"></p>
</body>
</html>
```

Operadores de comparación

Los operadores de comparación sirven para determinar la igualdad o la diferencia entre valores y variables

Se utilizan en las sentencias condicionales y tomar una decisión en función del resultado

```
if (age < 18) {text = "Too young"};
```

Operadores lógicos

Para poder realizar más una comparación

Operator	Description	Example
&&	and	(x < 10 && y > 1) is true
||	or	(x == 5 || y == 5) is false
!	not	!(x == y) is true

```
<html>
<body>
<p id="demo"></p>
<script>
var x=7;
var y=10
document.getElementById("demo").innerHTML=x>y;
</script>
</body></html>
```

Sentencias condicionales

En JavaScript tenemos las siguientes instrucciones condicionales:

- Utilizar **if** para especificar un bloque de código que se ejecutará si una condición específica es verdadera
- Utilice **else** para especificar un bloque de código que se ejecutará, si la misma condición es falsa
- Utilizar **else if** para especificar una nueva condición, si la primera condición es falsa
- Utilice **switch** para especificar muchos bloques de alternativas de código para ser ejecutado

Sentencia if

La sentencia **if** para especificar un bloque de código que se ejecutará si una condición específica es verdadera.

La sintaxis:

```
if (condition) {
   block of code to be executed if the condition is true
}
```

```
if (hour < 18) {
   greeting = "Good day";
}
```

```
<!DOCTYPE html>
<html>
<body>
<p>Display "Good day!" if the hour is less than 18:00:</p>
<p id="demo">Good Evening!</p>
<script>
if (new Date().getHours() < 18) {
document.getElementById("demo").innerHTML = "Good day!";
}
</script>
</body></html>
```

Sentencia else

Utilice **else** para especificar un bloque de código que se ejecutará, si la misma condición es falsa

La sintaxis:

```
if (condition) {
    bloque de código que se ejecutará si la condición es verdadera
} else { bloque de código que se ejecutará si la condición es falsa}
```

```
if (hour < 18) {
   greeting = "Good day";}
else {greeting = "Good evening";
}
```

```
<!DOCTYPE html>
<html>
<body>
<p>Click the button to display a time-based greeting:</p>
<button onclick="myFunction()">Try it</button>
<p id="demo"></p>
<script>
functionmyFunction() {
var hour = new Date().getHours();
var greeting;
if (hour < 18) {
greeting = "Good day";
   } else {
greeting = "Good evening";
   }
document.getElementById("demo").innerHTML = greeting;
}
</script>
</body>
</html>
```

JS puede cambiar la imagen de la bombilla

```
<h1>JavaScript puede cambiar la imagen</h1>
<img id="myImage" onclick="changeImage()" src="pic_bulboff.gif" width="100" height="180">
<p>Haga clic en la bombilla para encender o apagar la luz.</p>
<script>
function changeImage() {
var image = document.getElementById('myImage');
if (image.src.match("bulbon")) {
image.src = "pic_bulboff.gif";
   } else {
image.src = "pic_bulbon.gif";
}
}
</script>
```

Una variante del documento de la bombilla.

```
<script>
function light(sw) {
var pic;
if (sw == 0) {
pic = "pic_bulboff.gif"
   } else {
pic = "pic_bulbon.gif"
   }
document.getElementById('imagen').src = pic;
}
</script>
<img id="imagen" src="pic_bulboff.gif" width="100" height="180">
```

```
<button type="button" onclick="light(1)">Light On</button>
<button type="button" onclick="light(0)">Light Off</button>
```

Cambiar el estilo de un elemento HTML, es una variante de cambiar un atributo HTML:

```
<!DOCTYPE html>
<html>
<body>
<h1>¿Qué puede hacer JavaScript?</h1>
<p id="demo">JavaScript puede cambiar el estilo de un elemento .</p>
<script>
function myFunction() {
var x = document.getElementById("demo");
x.style.fontSize = "25px";
x.style.color = "red";
}
</script>
<button type="button" onclick="myFunction()">Haz click</button>
</body>
</html>
```

Sentencia else if

Utilizar **else if** para especificar una nueva condición, si la primera condición es falsa

La sintaxis:

if (*condition1*) {
  bloque de código que se ejecutará si condition1 es verdadero
} else if (*condition2*) {
  bloque de código que se ejecutará si la condición1 es falsa y la condición2 es verdadera
} else {
  bloque de código que se ejecutará si la condición1 es falsa y la condición2 es falsa
}

```
if (time < 10) {
  greeting = "Good morning";
} else if (time < 20) {
  greeting = "Good day";
} else {
  greeting = "Good evening";
}
```

```
<!DOCTYPE html>
<html>
<body>
<p>Click the button to get a time-based greeting:</p>
<button onclick="myFunction()">Try it</button>
<p id="demo"></p>
<script>
```

```
functionmyFunction() {
var greeting;
var time = new Date().getHours();
if (time < 10) {
greeting = "Good morning";
   } else if (time < 20) {
greeting = "Good day";
   } else {
greeting = "Good evening";
   }
document.getElementById("demo").innerHTML = greeting;
}
</script>
</body></html>
```

Sentencia switch

Utilice la sentencia **switch** para seleccionar uno de los muchos bloques de código para ser ejecutado.

```
switch(expression) {
   case n:
      code block
      break;
   case n:
      code block
      break;
   default:
      default code block
}
```

Así es como funciona:

- La expresión switch se evalúa una vez.
- El valor de la expresión se compara con los valores de cada caso.
- Si hay una coincidencia, se ejecuta el bloque asociado de código y salta a la siguiente sentencia de la llave.

El método **getDay()** devuelve el día de la semana como un número entre 0 y 6. (Domingo=0, Lunes=1,…

Utilice el número de días de la semana para calcular nombre del día

```
switch (new Date().getDay()) {
   case 0:
      day = "Sunday";
      break;
   case 1:
      day = "Monday";
      break;
   case 2:
      day = "Tuesday";
      break;
   case 3:
      day = "Wednesday";
```

```
      break;
   case 4:
      day = "Thursday";
      break;
   case 5:
      day = "Friday";
      break;
   case 6:
      day = "Saturday";
      break;
}
```

Cuando se ejecuta la orden o instrucción break se detiene case y salta a la instrucción siguiente de la llave

La palabra clave default especifica el código que se ejecutará si no hay ninguna coincidencia case

```
switch (new Date().getDay()) {
   case 6:
      text = "Today is Saturday";
      break;
   case 0:
      text = "Today is Sunday";
      break;
   default:
      text = "Looking forward to the Weekend";
}
```

Los bucles de JavaScript

Los bucles son útiles si se desea ejecutar el mismo código una y otra vez hasta que se cumpla la condición de salida.

A menudo este es el caso cuando se trabaja con arrays:

```
text += cars[0] + "<br>";
text += cars[1] + "<br>";
text += cars[2] + "<br>";
text += cars[3] + "<br>";
text += cars[4] + "<br>";
text += cars[5] + "<br>";
```

```
<!DOCTYPE html>
<html>
<body>
<p id="demo"></p>

<script>
var cars = ["BMW", "Volvo", "Saab", "Ford"];
var text = "";
var i;
```

```
for (i = 0; i<cars.length; i++) {
text += cars[i] + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>
</body>
</html>
```

JavaScript soporta<>tipos de bucles

Bucle for

El bucle **for** es una estructura de control en la que se puede indicar de antemano el número de repeticiones de un bloque de código

El bucle **for** tiene la siguiente sintaxis

for (*declaración 1*; *declaración 2*; *declaración 3*) {
  código de bloque para ser ejecutado
}

Declaración 1 se ejecuta ante del bloque de código. Se inicia la variable indice

Declaración 2 define la condición para ejecutar el bloque de código.

Declaración 3 se modifica la variable indice cada vez después de que el bloque de código se ha ejecutado

```
<script>
function myFunction() {
var text = "";
var i;
for (i = 0; i< 5; i++) {
text += "The number is " + i + "<br>";
  }
document.getElementById("demo").innerHTML = text;
}
</script>
```

```
<script>
var cars = ["BMW", "Volvo", "Saab", "Ford"];
var len=cars.lenght;
var text=”” ;
for (i = 0;i<len; i++) {
text += cars[i] + "<br>";
}
document.getElementById("demo").innerHTML = text;
</script>
```

La propiedad **for/in** recorre todos los atributos de un objeto

for (variable in objeto)sentencia

```
<!DOCTYPE html>
<html>
<body>
<p id="demo"></p>
<script>
var txt = "";
var person = {fname:"John", lname:"Doe", age:25};
var x;
for (x in person) {
txt += person[x] + " ";
}
document.getElementById("demo").innerHTML = txt;
</script>
</body>
</html>
```

El bucle While

El bucle **while** bucle recorre un bloque de código, siempre y cuando la condición especificada sea verdadera.

Sintaxis de la sentencia **while**:

while (*condition*) {
   code block to be executed
}

```
<script>
function myFunction() {
var text = "";
var i = 0;
while (i< 10) {
text += "<br>The number is " + i;
i++;
   }
document.getElementById("demo").innerHTML = text;
}
</script>
```

Ejemplo de un código JS utilizando **for** y utilizando **while**

```
var cars = ["BMW", "Volvo", "Saab", "Ford"];
var text = "";
```

```
for (i=0;i<cars.lenght;i++) {
   text += cars[i]  "<br>";
}

var cars = ["BMW", "Volvo", "Saab", "Ford"];
var i = 0;
var text = "";

while (i<cars.lenght) {
   text += cars[i] + "<br>";
   i++;
}
```

El bucle Do/While

El **do/while** es una variante del bucle while. Este bucle se ejecutará el bloque de código una vez, antes de comprobar si la condición es verdadera, entonces se repetirá el bucle mientras la condición es verdadera.

```
<script>
function myFunction() {
var text = ""
var i = 0;
do {
text += "<br>The number is " + i;
i++;
   }
while (i< 10)
document.getElementById("demo").innerHTML = text;
}
</script
```